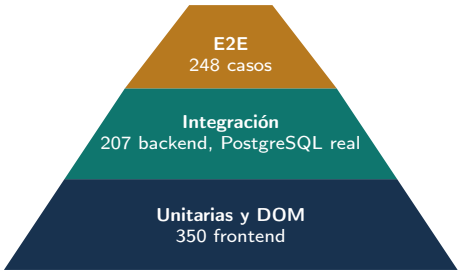
Estrategia de testing



- **Backend:** `node:test` contra PostgreSQL con RLS y el rol de runtime real; HTTP, roles y aislamiento UC/UTFSM.
- **Frontend:** `node:test` con DOM simulado; componentes, sesión y cliente de API.
- **E2E:** Playwright con API y base reales; 20 specs, 5 de seguridad (tenant, RBAC, auth, admin, auditoría). 13 min en CI.

Por qué así: lo más importante es asegurar que una institución no pueda acceder a los datos de otra y que las notas no sean visibles antes de publicarlas. Por eso realizamos pruebas con una base de datos real, además de las pruebas con mocks.

Qué probamos: flujos de usuario

Flujo	E2E	Integ. BE	Unit. FE	Qué asegura
Entrar a mi institución	44	7	5	Login global, sesión en memoria y una cuenta de UC que no ve UTFSM
Docente arma su curso	38	8	6	Creación atómica, secciones y asignar, revocar y reactivar roles
Docente publica material	16	7	9	Módulos ordenados, visibilidad y Markdown seguro
Docente crea y publica un quiz	18	7	11	Borrador validado, publicación irreversible y pauta protegida
Estudiante rinde el quiz	32	7	10	Guardar respuestas, envío atómico con autocorrección y cancelación
Notas: ponderar, publicar y consultar	42	12	11	Cálculo decimal, pesos congelados, nota visible solo al publicar y auditoría
Total	**248**	**66**	**55**	

E2E: fichas del catálogo; los totales suman además permisos (13), administración (11), red (13), accesibilidad (8), operación (9) y catálogo de API (4). Integración y unitarias: archivos de test; el resto es operación, seed y componentes compartidos.

Corrida completa del 31/10: 248 fichas, 512 ejecuciones en Chromium, Firefox y WebKit, 0 fallos